FINAL INTERNSHIP PROJECT

AI-Powered Personal Assistant

with n8n Workflow Automation

A Conversational, Multi-Tool Productivity Assistant for University Services

Domain	University Services/Personal Productivity
Frontend	Streamlit (Python)
Automation Layer	n8n
AI Model	Google Gemini
Organization	Quantum Synergy Solutions
Student Name	Zeeshan Ali
Submission Date	03-September-2026

1. Introduction

1.1 Project Title

AI-Powered Personal Assistant with n8n Workflow Automation

1.2 Problem Statement

University students and professionals routinely juggle several disconnected digital tools to manage their day-to-day responsibilities: a mail client for email, a calendar app for scheduling, a spreadsheet or budgeting app for expenses, a notes app for reminders, and a task manager for to-dos. Switching between these applications is time-consuming and breaks the user's focus. There is no single, natural-language interface through which a user can simply say what they want done and have it happen across all of these tools.

1.3 Objectives

- Build a conversational AI assistant that understands natural-language requests without requiring the user to pick a specific tool or menu.
- Use an AI Agent to detect user intent, choose the correct tool, extract the right parameters, and execute the action.
- Automate real-world actions (email, calendar, expense tracking, notes, tasks, and web search) through n8n workflows connected to Google services.
- Persist user data (expenses, notes, tasks) in real storage rather than relying only on conversational memory.
- Return clean, human-readable responses rather than raw API output or internal reasoning.
- Design the system so additional tools and services can be added later without restructuring the architecture.

2. Proposed Solution

2.1 System Description

The proposed system is a single, centralized Personal AI Assistant. A user types a natural-language message into a Streamlit chat interface. The message is sent to an n8n webhook, where an AI Agent (powered by Google Gemini) interprets the request, decides whether a tool is needed, executes the appropriate tool or combination of tools, and returns a concise natural-language answer. The Streamlit frontend then displays that answer in a conversational, chat-style layout.

Unlike a menu-driven assistant, the user never has to choose a category such as "Gmail" or "Calendar" — the AI Agent infers the correct action directly from the wording of the request, and can chain multiple tools together for compound requests.

2.2 Selected Domain

The assistant is framed around University Services / personal productivity, following the same pattern as the reference "AI Student Assistant" example: a student can ask questions, retrieve information, and have the assistant carry out real actions (sending emails, scheduling events, logging expenses, saving notes, and creating tasks) on their behalf.

2.3 Main Features

- Conversational chat interface (Streamlit) with persistent chat history.
- Natural-language intent detection and automatic tool selection (no manual tool picking).
- Gmail integration — read, search, and send emails.
- Google Calendar integration — create events and retrieve single or multiple events.
- Expense tracking — add, retrieve, update, and calculate expenses stored in Google Sheets.
- Notes management — create, update, and retrieve notes.
- Google Tasks integration — create and retrieve tasks.
- Web search tool for current/external information not available in the user's connected accounts.
- Multi-tool orchestration for compound requests (e.g. "check tomorrow's calendar and email the person I'm meeting").
- Short-term conversational memory so follow-up messages are understood in context.

3. System Design

3.1 System Architecture

The system follows a clean separation between presentation, orchestration, and external services:

Streamlit Frontend (Chat UI)

↓

Webhook (HTTPS POST)

↓

n8n AI Agent (Google Gemini + Simple Memory)

↓

Tool Layer: Gmail · Calendar · Expenses · Notes · Google Tasks · Web Search

↓

Google Services / External APIs

↓

Respond to Webhook

↓

Streamlit Frontend (Response Displayed)

The Streamlit layer is responsible only for the user interface and for sending/receiving JSON over the webhook — it contains no business logic for Gmail, Calendar, Expenses, Notes, or Tasks. All orchestration, tool selection, and execution logic lives inside the n8n AI Agent, keeping the two layers independently maintainable.

3.2 Data Flow

Request	Streamlit sends {"message": "user's request"} to the n8n webhook.
Understanding	The AI Agent (Gemini) performs intent detection and parameter extraction.
Tool Selection	The Agent selects the minimum number of tools needed (single or multiple).
Execution	Selected tool(s) call Gmail, Calendar, Google Sheets (expenses/notes), Google Tasks, or Web Search.
Memory	Simple Memory retains recent conversational context (e.g. a follow-up expense entry).
Response	The Agent formats a natural-language answer; n8n returns {"response": "assistant's final answer"}.
Display	Streamlit renders the answer in the chat window.

3.3 Intent Categories

- chat — general questions answered directly by the model.
- web_search — current or external information lookups.
- gmail — reading, searching, or sending email.
- calendar — creating or retrieving calendar events.
- expenses — adding, retrieving, updating, or totalling expenses.
- notes — creating, updating, or retrieving notes.
- tasks — creating or retrieving Google Tasks.
- multi_tool / complex — requests that require chaining two or more tools.

4. Implementation

4.1 AI Agent

The AI Agent is implemented as an n8n AI Agent node using Google Gemini as its chat model, with a Simple Memory node attached for short-term conversational context. The agent's system prompt instructs it to: understand the request, decide whether a tool is required, select the correct tool(s), supply correct parameters, execute the tool(s), interpret the results, and produce a single concise natural-language reply. The agent never exposes its internal reasoning, tool names, or raw API/JSON responses to the user — for example, a request for expenses returns a short bulleted summary with a total, not the underlying spreadsheet rows.

Tools connected to the AI Agent:

- Web Search (Google Search / SerpApi) — current information and research.
- Gmail — Get Single Message, Get Many Messages, Send a Message.
- Google Calendar — Create Event, Get Single Event, Get Many Events.
- Expenses — Calculator, Add Expense, Get Expenses, Update Expense.
- Notes — Create Note, Update Note, Get Notes.
- Google Tasks — Create Task, Get Single Task, Get Many Tasks.

4.2 n8n Workflow

The core n8n workflow consists of three main stages: a Webhook node that receives the request from Streamlit, the AI Agent node that processes the request end-to-end using the tools listed above, and a Respond to Webhook node that returns the final answer as JSON. Each tool category (Gmail, Calendar, Expenses, Notes, Tasks) is implemented as its own set of n8n tool nodes wired into the AI Agent, keeping tool-specific logic isolated and easy to extend.

4.3 Data Storage

Expenses and notes are stored in Google Sheets, acting as lightweight, beginner-friendly persistent storage that both the Add/Update and Get tools read from and write to. Tasks and calendar events are stored natively in the user's Google Tasks and Google Calendar accounts. Conversational memory (Simple Memory) is used only to retain short-term context between turns and is never treated as a substitute for this persistent storage.

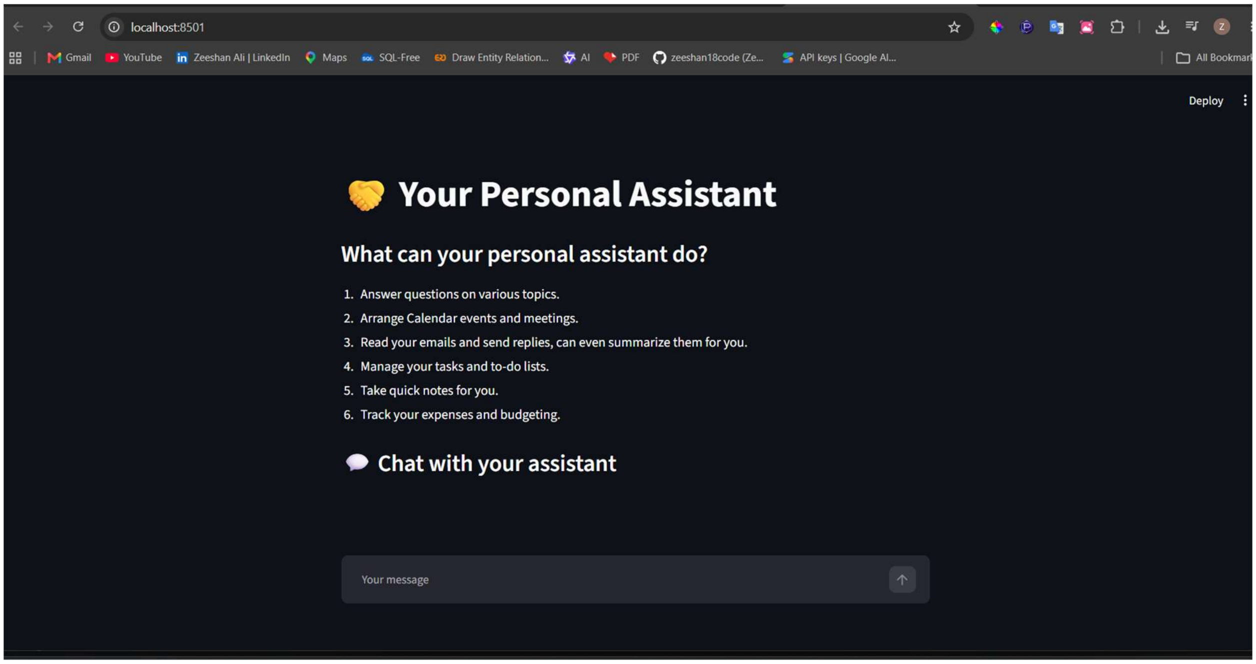
4.4 Frontend Implementation

The Streamlit application presents a chat-style interface where the user types a message, the app posts it to the n8n webhook, waits for the JSON response, and displays the assistant's reply alongside the running conversation history. The frontend handles connection or timeout errors gracefully with a user-friendly message, and does not expose API keys, credentials, or internal n8n implementation details to the user.

5. Results

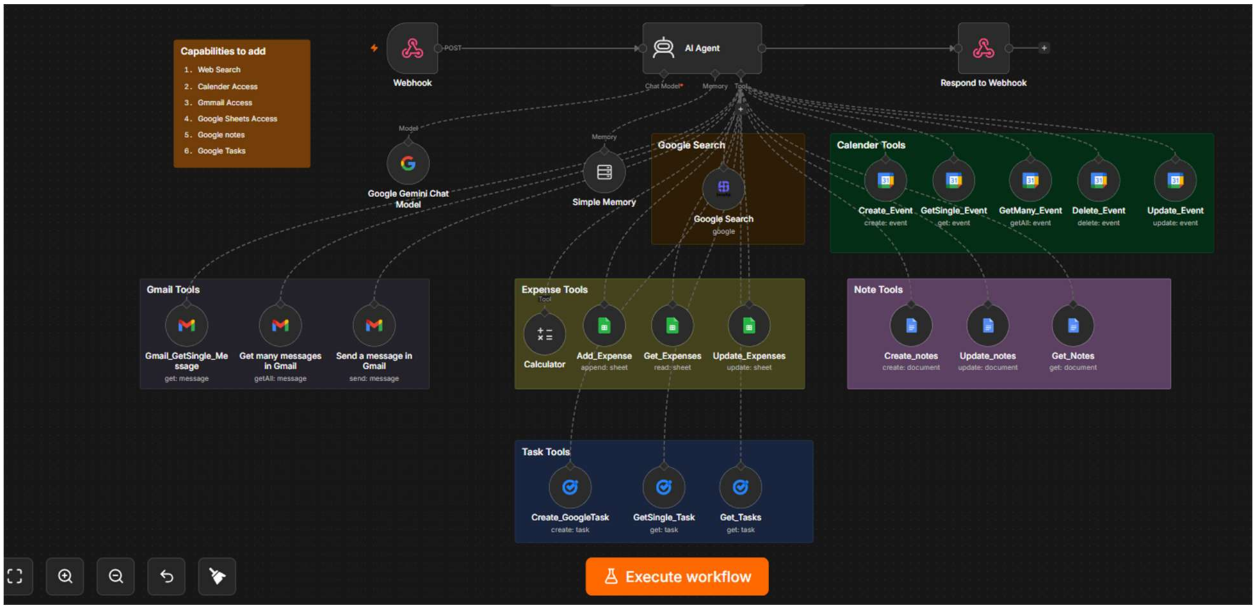
The following screenshots demonstrate the completed system end to end, from user input through to the automated action and final response.

5.1 User Interface



5.2 AI Agent

5.3 n8n Workflow

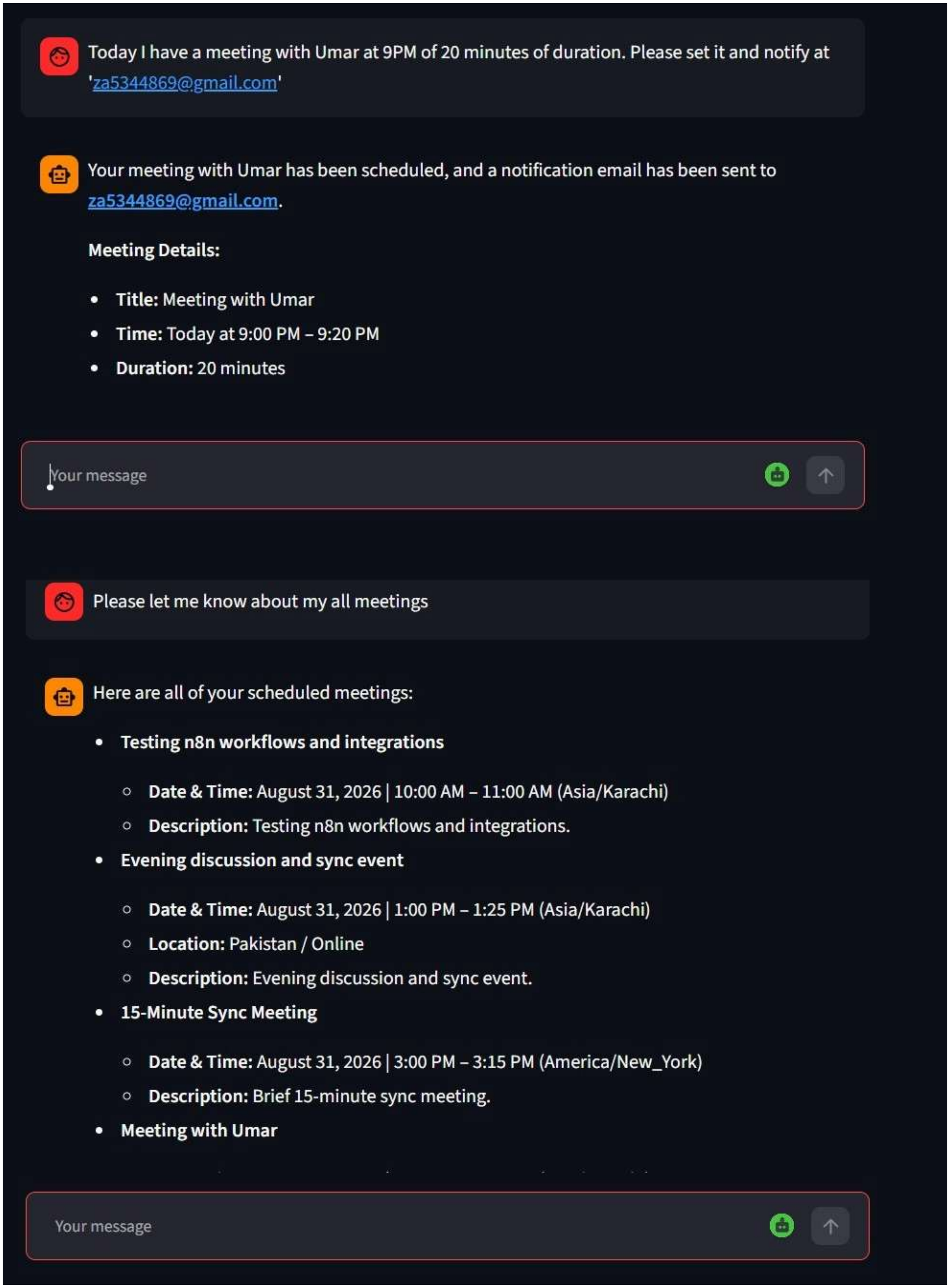


5.4 Google Sheet

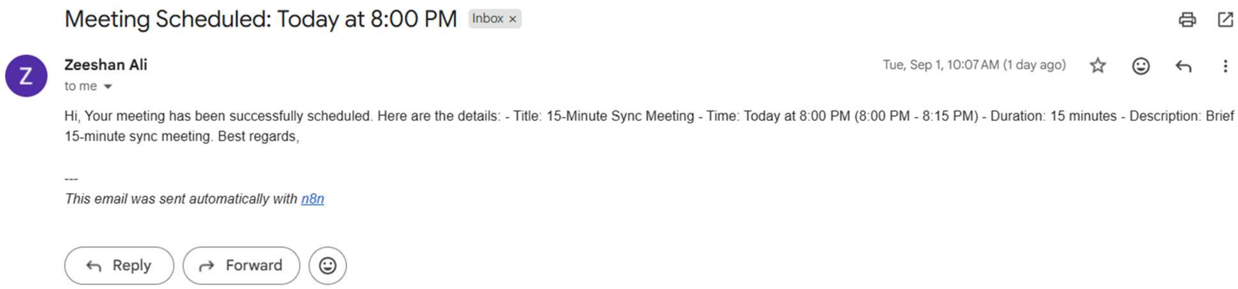
A1 fx Id

	A	B	C	D	E	F
1	Id	Date	Expense_Category	Expense	Remarks	
2	2	2026-08-31	Utilities	1400 PKR	Electricity bill	
3	1	2026-08-31	Groceries	400 PKR	milk	
4	3	2026-08-31	Utilities	350 PKR	Yogurt	
5	4	2026-08-31	Groceries	300 PKR	milk	
6	5	2026-08-31	Sports	1500	Cricket Bat	
7						
8						

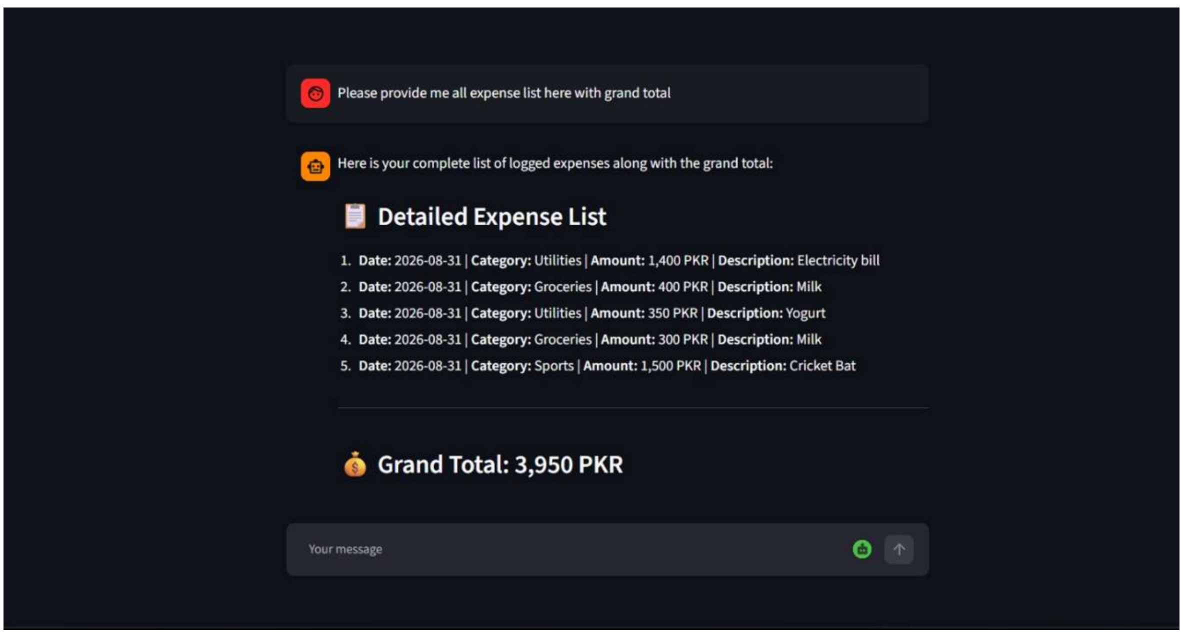
5.5 Example Inputs and AI Agent Responses



5.6 Email Notification



5.7 Final System Output



6. Conclusion

6.1 What Was Achieved

The project successfully delivers a working, centralized Personal AI Assistant that understands natural-language requests and carries out real actions across Gmail, Google Calendar, expense tracking, notes, and Google Tasks, in addition to answering general questions and performing web searches. The AI Agent reliably distinguishes between single-tool and multi-tool requests, and returns clean, human-readable responses instead of raw tool output. The Streamlit-to-n8n webhook integration provides a simple, responsive chat experience while keeping all automation logic centralized in n8n.

6.2 Limitations

- The assistant depends on free-tier API/model rate limits (Gemini, Google Sheets, Gmail), which can throttle heavy usage.
- Simple Memory provides only short-term conversational context and is not a substitute for a full database or user-account system.
- There is currently no authentication layer, so the assistant is scoped to a single user's connected Google account.
- Complex multi-step requests occasionally require more precise phrasing for the Agent to extract all needed parameters correctly.

6.3 Future Improvements

- Add authentication and multi-user account support.
- Move from Google Sheets to a persistent database (e.g. PostgreSQL or MongoDB) as usage scales.
- Expand tool coverage: Google Drive, Google Keep, advanced Gmail operations, and file processing.
- Add voice input/output and push notifications.
- Introduce specialized sub-agents for domains such as expenses or scheduling to improve accuracy on complex requests.
- Add automated testing and monitoring around n8n workflow reliability and API error handling.